

Retrieval-Augmented Generation (RAG)

A Fully Written 10-Page Practical Guide

From document loading and chunking to embeddings, retrieval, LangChain, Ollama, advanced retrieval, evaluation, and production design.

1. What Is Retrieval-Augmented Generation?

Retrieval-Augmented Generation, commonly called RAG, is a way of building applications in which a large language model is given relevant external information before it generates an answer. The important idea is that the model does not have to rely only on knowledge stored in its parameters. Instead, a separate retrieval system searches a collection of documents and places the most useful pieces of those documents into the model's context.

Imagine that you are building a chatbot for a university. The chatbot needs to answer questions about attendance rules, examination regulations, placement policies, course syllabi, and hostel rules. A general-purpose language model may know what a university is, but it cannot be expected to know the current rules of your particular university. You could try to train a model on all of these documents, but that would be expensive and would make updating the information difficult. RAG provides a more practical solution: store the documents in a searchable knowledge base, retrieve relevant passages when a question arrives, and ask the language model to answer using those passages.

The phrase Retrieval-Augmented Generation describes the three conceptual stages. Retrieval finds useful evidence. Augmentation places that evidence into the model's input. Generation uses the language model to produce a natural-language response.

A simplified flow is:

User question → Retriever → Relevant document chunks → Prompt with context → LLM → Answer

RAG is therefore not a single model. It is a system made from multiple components. Document loaders, text splitters, embedding models, vector stores, retrievers, prompts, language models, and output parsers can all participate in the system.

One of the most important ideas to remember is that RAG does not magically give an LLM new permanent knowledge. The retrieved information is supplied at inference time. If the underlying documents are updated, the retrieval index can also be updated without retraining the language model.

2. Why Do We Need RAG?

Large language models are trained on enormous amounts of text and learn patterns about language, facts, reasoning, and programming. However, their internal knowledge has several limitations. First, the model's training data has a cutoff or finite collection date. Second, a model normally does not have access to an organization's private documents. Third, even when a model has learned a fact, it may produce an incorrect answer because generation is based on learned patterns rather than a guaranteed database lookup.

RAG addresses these limitations by separating general language ability from external knowledge. The language model can remain responsible for understanding questions and writing answers, while the retrieval system is responsible for finding evidence.

Consider a company support assistant. The company may have thousands of pages of internal documentation. These documents may change every week. If the password-reset policy changes, it would be inconvenient to retrain a language model. With RAG, the company can update the source document and re-index the changed content. A future question can then retrieve the new policy.

RAG is also useful for reducing hallucination. A hallucination occurs when a model generates information that is unsupported or incorrect. Retrieval gives the model actual passages that it can use as evidence. This does not guarantee a correct answer, because the retrieved passages themselves may be wrong or irrelevant, but it provides a much stronger basis for generation.

Another advantage is traceability. If a RAG application returns citations, a user can inspect the source passages behind an answer. This is especially valuable in technical, business, academic, and enterprise environments.

RAG and fine-tuning should not be confused. Fine-tuning changes model behavior by training on additional examples. RAG changes the information available to the model at inference time. If the main problem is changing factual knowledge, RAG is often more suitable. If the main problem is consistent style, behavior, or task-specific output patterns, fine-tuning may be useful.

The strongest systems can combine both approaches. A specialized model can be fine-tuned for a particular task while RAG supplies current and private knowledge.

3. The Complete RAG Pipeline

A practical RAG application normally has an indexing pipeline and a query pipeline. Understanding this separation makes the architecture much easier to understand.

The indexing pipeline runs before users ask questions. It starts with source documents. These may be PDF files, text files, web pages, Word documents, databases, or other sources. A document loader reads each source and converts it into a common representation. The documents are then cleaned and divided into smaller pieces called chunks.

Each chunk is passed to an embedding model. The embedding model converts the text into a vector, which is a numerical representation of its semantic meaning. These vectors and their associated text and metadata are stored in a vector database or another retrieval index.

The query pipeline begins when the user asks a question. The question is converted into an embedding, and the retrieval system searches for document chunks whose vectors are similar to the query. The best chunks are selected and placed into a prompt together with the user's question.

The language model then reads the question and retrieved context and generates the final answer.

The complete conceptual architecture is:

Documents → Document Loader → Text Splitter → Chunks → Embedding Model → Vector Store

User Question → Query Embedding → Retriever → Relevant Chunks → Prompt → LLM → Answer

Each component has a specific job. The loader handles source extraction. The splitter controls how information is divided. Embeddings provide a semantic representation. The vector store makes similarity search efficient. The retriever chooses evidence. The prompt tells the model how to use that evidence. The LLM turns evidence into a useful response.

This separation is important because failures can occur at different stages. If the correct document is never retrieved, changing the generation prompt may not solve the problem. If retrieval is excellent but the model ignores the context, generation or prompting may be the bottleneck.

For this reason, a serious RAG project should evaluate retrieval and generation separately rather than treating the entire application as one black box.

4. Document Loaders and Document Objects

The first practical step in many RAG projects is getting information out of its original format. A document loader performs this task. Different loaders are designed for different source types because a PDF, web page, CSV file, and plain text file do not store information in the same way.

In LangChain, a loader generally produces Document objects. A Document commonly contains two important fields: `page_content` and `metadata`. `page_content` contains the actual text. `metadata` contains information about where the text came from or other properties that may be useful later.

For example, a text file can be loaded like this:

Practical example:

```
from langchain_community.document_loaders import TextLoader

loader = TextLoader("example.txt")
docs = loader.load()

print(docs[0].page_content)
print(docs[0].metadata)
```

5. Text Splitting and Chunking

A complete document is usually too large to send to an LLM every time a user asks a question. More importantly, retrieving an entire document when only one paragraph is relevant wastes context and can reduce answer quality. Therefore, RAG systems normally split documents into chunks.

A chunk is a smaller section of a document that can be independently retrieved. Chunking is not simply a technical preprocessing step; it has a direct effect on retrieval quality.

Suppose a document contains a heading followed by five paragraphs explaining a policy. If chunks are too small, the heading might be separated from the explanation. A retrieved paragraph could then be technically relevant but difficult to interpret. If chunks are too large, a query about one sentence may retrieve several pages containing unrelated information.

Chunk size and overlap are common parameters. Chunk size controls how much text belongs to each chunk. Overlap repeats a small amount of text between neighboring chunks. Overlap helps preserve information that crosses a chunk boundary.

There is no universal perfect chunk size. A good configuration depends on the documents and questions. Technical documentation may benefit from structure-aware splitting. Legal documents may need sections and clauses to remain intact. Source code may need function or class boundaries. Markdown can often be split using headings.

A common beginner pipeline is:

Document → Recursive Text Splitter → Chunk 1, Chunk 2, Chunk 3, ...

The important lesson is to preserve meaning, not merely to produce equal-sized strings.

6. Embeddings and Vector Databases

After documents have been split into chunks, the system needs a way to search them by meaning. Embeddings provide this capability. An embedding model takes text and maps it to a numerical vector.

For example, these two sentences use different words:

'How long does the battery last?' 'What is the operating time on a full charge?'

A good embedding model can represent these sentences as semantically related even though they do not share many exact words.

The resulting vector may contain hundreds or thousands of numerical dimensions. The values themselves are not normally interpreted by a human. Instead, the vectors are compared using a similarity or distance function.

A vector database stores these vectors together with the original chunks and metadata. When a user submits a query, the query is embedded and compared with stored vectors. The system returns the closest candidates.

Common similarity approaches include cosine similarity, dot product, and Euclidean distance. The appropriate method depends on the embedding model and vector database.

A stored record conceptually looks like:

Embedding vector + text chunk + metadata

Metadata is important because retrieval does not always need to be purely semantic. Imagine a company has documents for several departments. A user may ask a question about the Finance department. Metadata filters can restrict retrieval to Finance documents before or alongside semantic search.

Vector databases therefore act like semantic indexes. They do not replace the LLM. Their job is to help the application find useful evidence quickly.

7. Retriever, Prompt, and LLM

A retriever is the component that takes a query and returns relevant documents. A simple retriever may perform vector similarity search and return the top-k chunks. More advanced retrievers can combine vector search with keywords, metadata filters, query rewriting, or other techniques.

The value of top-k is important. If k is too small, the system may miss useful evidence. If k is too large, the prompt may contain many irrelevant passages. The correct value should be determined through evaluation rather than chosen blindly.

Once the relevant documents have been retrieved, they must be inserted into a prompt. A RAG prompt usually contains instructions, context, and the user's question.

For example:

Practical example:

```
prompt = PromptTemplate.from_template("""
Answer the question using only the context below.

Context:
{context}

Question:
{question}

If the context does not contain the answer, say that
the information is not available in the provided context.
""")
```


8. Building RAG with LangChain and Ollama

LangChain provides a convenient way to connect RAG components. A simple chain can use a prompt, a chat model, and an output parser. The pipe operator is used to compose these components.

If Ollama is used for local inference, the model can run on the user's machine rather than being sent to a hosted model provider. This can be useful for experimentation and privacy-sensitive applications.

A simple summarization chain over a loaded document can be written as:

Practical example:

```
from langchain_ollama import ChatOllama
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import PromptTemplate

model = ChatOllama(model="llama3.1:latest")

prompt = PromptTemplate.from_template(
    "Summarize this document in two lines:\n{paragraph}"
)

chain = prompt | model | StrOutputParser()

result = chain.invoke({
    "paragraph": docs[0].page_content
})

print(result)
```

9. Advanced RAG: Hybrid Search, Reranking, and Query Transformation

Basic vector retrieval is a useful starting point, but real applications often need more sophisticated retrieval. One reason is that semantic similarity and exact keyword matching solve different problems.

Hybrid search combines dense retrieval with lexical retrieval. A keyword system can be very good at finding exact identifiers such as error codes, product numbers, names, or specialized terms. Dense retrieval is good at finding conceptually related passages even when the wording differs. Combining them can make retrieval more robust.

Reranking is another common technique. Instead of sending the first retrieved results directly to the LLM, the system retrieves a larger candidate set and then uses a reranker to score the query-document relationships more carefully. Only the best passages are passed to the generation model.

Query transformation changes the query before retrieval. A conversational question may need to be rewritten into a standalone query. Multi-query retrieval can generate several formulations of the same question and search with each one. Query decomposition can split a complex question into smaller questions.

These methods improve retrieval in different ways, but they also add latency and complexity. Advanced RAG should therefore be introduced when evaluation shows that the basic retriever is not good enough.

10. Evaluation, Hallucination, and Production RAG

A RAG application should not be judged only by whether the final answer sounds fluent. A polished answer can still be wrong if the retrieval system supplied the wrong evidence.

Evaluation should therefore examine at least two layers: retrieval and generation. Retrieval evaluation asks whether the relevant evidence was found. Metrics such as recall@k and precision@k can be used when suitable ground truth is available. Generation evaluation asks whether the answer is correct, relevant, complete, and grounded in the retrieved context.

A useful evaluation dataset contains realistic user questions, expected evidence, and expected answer properties. It should include straightforward questions, ambiguous questions, questions whose answers are absent from the knowledge base, and questions requiring information from multiple passages.

Hallucination is another important concern. RAG reduces hallucination risk but does not eliminate it. The retriever can return irrelevant text, documents can contain contradictions, and the model can still generate unsupported claims.

A good RAG system should be able to abstain. If the retrieved context does not contain enough information, the model should say that the available sources are insufficient rather than inventing an answer.

Production RAG also requires security and operational controls. Access permissions must be respected during retrieval. A vector database should not accidentally expose confidential documents to users who are not authorized to see them. Documents need to be updated when their source changes, and embedding or chunking changes may require re-indexing.

Latency, cost, logging, monitoring, caching, retries, rate limits, and failure handling become important as usage increases.

The best way to learn RAG is to build it incrementally. Start with a text file and `TextLoader`. Add a text splitter. Create embeddings. Store chunks in a vector store. Build a retriever. Connect retrieval to a prompt and an LLM. Then add citations, evaluation, reranking, hybrid search, and production controls.

The most important mental model is this: RAG is a system that gives an LLM relevant external evidence at inference time. Retrieval determines what the model gets to see, while generation determines how the model uses that evidence. If you understand both sides, you understand the core of RAG.